# Exact and Heuristic Methods for Capacitated Path Planning for Agricultural Field Operations

Shahadat Hossain
University of Northern British Columbia
Prince George, BC, Canada
shahadat.hossain@unbc.ca

Fabliha Zahin
University of Lethbridge
Lethbridge, AB, Canada
fabliha.zahin@uleth.ca

## Abstract

This paper addresses the Capacitated Coverage Path Planning Problem (CPP) arising in agricultural field operations. It aims to determine an efficient sequence of field tracks to be serviced by an agricultural machine subject to limited onboard capacity and refilling constraints, while minimizing non-working distance. The non-working distance is the length traveled by the machinery not performing productive fieldwork. Optimizing how agricultural machinery maneuvers can help reduce operational costs and greenhouse gas (GHG) emissions, while increasing productivity. The problem is formulated as a variant of the constrained Vehicle Routing Problem (VRP), which is NP-hard. A novel exact Integer Linear Programming (ILP) formulation is proposed. We employ a commercial optimizer, Gurobi, to obtain optimal reference solutions for smaller instances. To overcome scalability limitations, a metaheuristic based on "simulated annealing (SA) methodology" is proposed. Our algorithm incorporates customized neighborhood search operators and a capacity-aware route-splitting mechanism to explicitly handle refilling operations. Computational experiments are conducted on real-world field instances. The proposed metaheuristic is evaluated against the exact formulation. The results indicate that SA consistently produces near-optimal solutions with substantially reduced computation times.

## CCS Concepts

• **Applied computing** → **Multi-criterion optimization and decision-making**; • **Theory of computation** → *Theory of randomized search heuristics.*

## Keywords

Agricultural Field Logistics, Coverage Path Planning, Capacitated Routing, Metaheuristics, Simulated Annealing, Integer Linear Programming, Optimization

**ACM Reference Format:**
Shahadat Hossain and Fabliha Zahin. 2026. Exact and Heuristic Methods for Capacitated Path Planning for Agricultural Field Operations. In *Genetic and Evolutionary Computation Conference (GECCO '26), July 13–17, 2026, San Jose, Costa Rica.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3795095.3805120



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
*GECCO '26, San Jose, Costa Rica*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2487-9/2026/07
https://doi.org/10.1145/3795095.3805120

## 1 Introduction

Modern agricultural field operations such as harvesting, spraying, and fertilization require machinery to repeatedly traverse a set of predefined field tracks. Beyond productive traversal, machines incur substantial *non-working distance* due to headland maneuvers, travel between tracks, and trips to refill stations when onboard capacity is limited. Reducing this non-productive motion directly improves operational efficiency and is especially important when planning sustainable autonomous or semi-autonomous operations. This, in turn, mitigates sustainability concerns like fuel consumption and GHG emissions [4][3] and soil compaction by avoiding unnecessary wheel passes [14].

The problem formulated is closely related to classical routing and coverage formulations, including the Vehicle Routing Problem (VRP) and its capacitated variants, like Capacitated Vehicle Routing Problem (CVRP) and Capacitated Arc Routing Problem (CARP). Thus, the problem inherits NP-hardness from these classes [19][21]. The Vehicle Routing Problem (VRP) is a fundamental problem in combinatorial optimization and operations research. Dantzig and Ramser first talked about it in 1959 as a way to improve fuel delivery routes [9]. The VRP is a more general version of the Traveling Salesman Problem (TSP) that works with more than one vehicle and serves customers in different locations. The goal is to lower the total cost of travel while making sure that each customer is only visited once and that every route starts and ends at the depot. Braekers, Ramaekers, and Van Nieuwenhuys (2016) present an extensive taxonomy of VRP variants and solution paradigms, categorizing more than 270 studies from 2009 to 2015 [5]. The Capacitated Vehicle Routing Problem (CVRP) is one of the most important variants of VRP. In the CVRP, each vehicle can only carry a certain amount of goods or materials, and deliveries of demands must be made without going over that limit. The CVRP is still an NP-hard problem, and it has been shown to be APX-hard in general metric spaces [6]. State-of-the-art exact methods use integer linear programming formulations that are solved with "branch-and-cut" and "branch-and-cut-and-price" algorithms. These algorithms add valid inequalities and create columns to efficiently represent routes. Bard, Kontoravdis, and Yu (2002) illustrated the efficacy of branch-and-cut for the Vehicle Routing Problem with Time Windows (VRPTW) [2]. More recent frameworks, like Pessoa et al. (2018), combine branch-and-cut with route pricing to find the best solution for medium-sized CVRP problems, filling in gaps that had been open for decades [18]. These approaches guarantee global optimality but suffer exponential growth in runtime with increasing instance size, limiting practicality for large-scale or real-time applications. The Capacitated Arc Routing Problem (CARP) puts demand on edges (like road segments), which makes it better for

services like street cleaning or waste collection. Agricultural field path planning had previously been modeled using the Capacitated Arc Routing Problem, notably by Khajepour et al. who solve the problem using an Adaptive Large Neighbourhood Search (ALNS) heuristic [15].

This paper studies a refill-aware track servicing problem that we refer to as the *Capacitated Coverage Path Planning* problem (Capacitated-CPP). The task is to determine (i) an ordered sequence of track segments to service, (ii) a traversal direction for each track, and (iii) a partition of the service sequence into depot-to-depot tours (refill cycles), so that capacity constraints are respected and total non-working distance is minimized. In practice, exact optimization can provide strong guaranty of solution quality, but its computational cost can grow quickly with instance size and route-structure constraints. A key distinction from prior CARP-based agricultural planning approaches such as [15] is the strict enforcement of single traversal of working tracks, motivated by the need to reduce soil compaction.

To capture the problem precisely and enable benchmarking, we first present an Integer Linear Programming (ILP) model that yields optimal solutions for small instances. We develop a Simulated Annealing (SA) metaheuristic tailored to the refill-aware structure of field logistics. The proposed SA includes problem-specific neighbourhood moves and a capacity-aware route splitting mechanism that explicitly models refilling actions.

The main contributions of this work include a compact ILP formulation for the capacitated field CPP, with a novel approach at isolating and minimizing the non-working distance as the objective function, a problem-aware SA metaheuristic with customized neighbourhood search operators and capacity-aware splitting to handle refilling constraints efficiently and an empirical evaluation on real-world benchmark instances showing that SA produces near-optimal solutions with substantially reduced runtime compared to exact optimization. This research was conducted in collaboration with Verge Ag, which is a Canadian agri-tech company developing digital tools for operational planning. This partnership ensured that the proposed methods were evaluated on realistic datasets and problem setting directly relevant to precision agriculture.

The remainder of this paper is organized as follows. Section 2 introduces the capacitated coverage path planning problem, reviews the related literature, and presents a novel integer linear programming (ILP) formulation. Section 3 describes a simulated annealing–based solution approach, followed by a splitting procedure that decomposes the resulting "flattened track sequence" into feasible tours. Section 4 reports numerical results obtained from three real-world agricultural fields with varying levels of geometric complexity. Finally, Section 5 concludes the paper and outlines directions for future research.

## 2 Capacitated Coverage Path Planning

Agricultural field operations are a specific type of routing and coverage problem in which machines must travel all of the working tracks of a field while taking into account the field's layout, the ability to make turns, and the limited capacity. While coverage path planning (CPP) has been extensively examined in mobile robotics [17][7][12], the explicit formulation of in-field machine movement as a Vehicle Routing Problem (VRP) is a recent development. Bochtis and Sørensen [4][3] proposed the Vehicle Routing Problem in Field Logistics (VRPFL) framework, which turns the physical task of servicing an agricultural field into a graph-based routing problem. This abstraction shows the field layout as a directed graph $G = (V, A)$, where each working track is a "customer" node and the arcs are the possible headland connections between tracks. The "depot" is the field entrance or service trailer. Each track has a demand value, which is the amount of input material used or crop collected. The vehicle's limited capacity $Q$ limits how many tracks it can service before going back to the depot. The optimization looks for a set of tours that cover all the tracks exactly once and travel the least amount of non-operational distance.

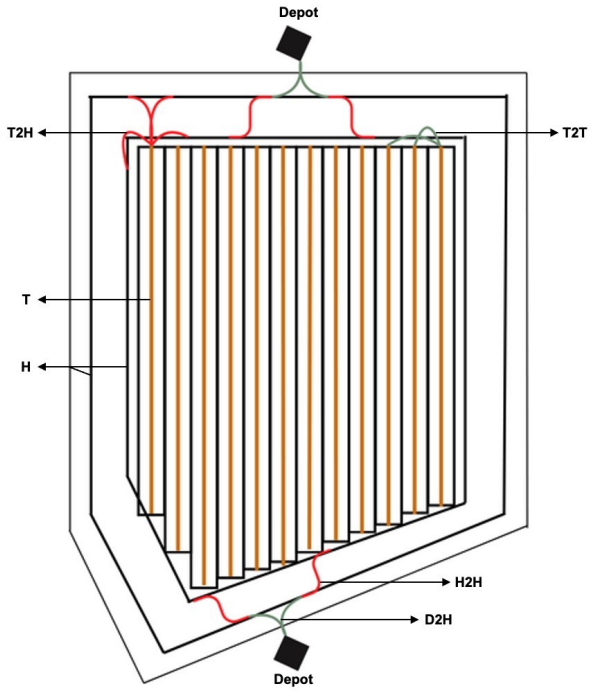



**Figure 1: An illustrative field topology used to represent tracks, headlands, and depot connections adapted from Vahdanjoo *et al.*, 2020 [20].**

Figure 1 shows all the possible connecting paths in the geometric representation of a simple field where **T** represents a workable track, and **H** is a feasible headland pass. A route that connects the ends of fieldwork tracks to the ends of nearby fieldwork tracks is denoted by **T2T**, a route that goes from each depot to the nearest headland pass is denoted by **D2H**, a route that goes from one headland pass to the next is given by **H2H**, and a route that connects the ends of fieldwork tracks leading to each headland pass is given by **T2H**.

## 2.1 Problem Modelling

In our work we model the capacitated coverage path planning problem as a directed graph

$$G = (V, A)$$

where:

- $V$ is the set of nodes, representing the depot and the two endpoints of each field track
- $A = V \times V$ is the set of feasible directed arcs connecting these nodes.

Each track $i \in \{1, \ldots, n\}$ stands for a single working pass of the machine (e.g., spraying, fertilizing, harvesting). Its two physical endpoints are indexed as nodes $(2i-1)$ and $(2i)$. A *working arc* is the distance between two endpoints on the track. For each track we include both directions $(2i-1, 2i)$ and $(2i, 2i-1)$ in the model. However, only one will be chosen in the final solution so that every track is covered exactly once.

The depot node represents the out-of-field service location (refilling or unloading station). A tour, which is one refill cycle that services a subset of tracks without exceeding the vehicle capacity, starts and ends at the depot. Movement between track endpoints along headlands/boundaries is shown by *non-working arcs*. Accordingly, the arc set is divided into two parts:

$$A \;=\; A_t \;\cup\; A_0,$$

where $A_t$ are working (track) arcs and $A_0$ are non-working arcs (headland transitions, repositioning). Each arc $(i, j) \in A$ is associated with a travel cost $c_{ij}$ (e.g., Euclidean or measured path length). Only non-working travel $A_0$ is penalized in the objective; distance along $A_t$ represents essential productive work. Each machine has finite capacity $Q$ (e.g., tank volume or grain storage). Each track $i$ has a demand $d_i$ (material applied/harvested when that track is serviced). Each vehicle completes a *tour* $k$, visiting a subset of tracks so that the total serviced demand does not exceed the capacity $Q$, and only goes through feasible headland (non-working) arcs between track endpoints. If $S_k \subseteq \{1, \ldots, n\}$ is the set of tracks serviced on tour $k$, then

$$\sum_{i \in S_k} d_i \;\le\; Q.$$

Because the total field demand is usually more than $Q$, several depot-to-depot tours might be needed. Consequently, the proposed formulation is structurally linked to the Capacitated Arc Routing Problem (CARP) [8], yet it incorporates several essential modifications tailored to agricultural field logistics.

First, the underlying graph separates *working* arcs (on-track traversal) from *non-working* or *headland* arcs. The objective function only penalizes the latter. This separation is based on the fact that productive passes help cover the field and do not make travel less efficient. Second, connectivity is limited to headland arcs, which show possible machine transitions between track endpoints. Third, vertices do not represent intersection points in a road network as in CVRP. Instead, they represent physical track endpoints based on the field layout. This makes sure that routing decisions map machine movements directly (i.e. how the machines enter, exit and transitions between tracks). Finally, the capacity constraint clearly shows the limits on material refills (like the capacity of a tank), with depot returns standing in for refilling operations. These changes create a model called Capacitated Coverage Path Planning (CPP) that takes into account the geometry and operational limits of field logistics while still being closely related to the classical CARP formulation framework.

## 2.2 Mathematical Formulation (ILP)

We define the Capacitated Coverage Path Planning Problem as an integer linear program (ILP) that minimizes non-working travel distance while adhering to routing, coverage, and capacity constraints.

Let $K$ denote the set of tours. Given per-track demands $d_i$ and capacity $Q$, any feasible solution must use at least

$$K_{\min} = \left\lceil \frac{\sum_{i=1}^{n} d_i}{Q} \right\rceil$$

tours. Therefore, in the ILP implementation we set $|K| = K_{\min}$ (and, if an instance is infeasible due to arc feasibility/connectivity, we increment $|K|$ and re-solve). Thus, $|K|$ is determined by the instance demands and capacity rather than chosen arbitrarily. For each directed arc $(i, j) \in A$ and tour $k \in K$, binary decision variables are defined as:

$$x_{ij}^k = \begin{cases} 1, & \text{if tour } k \text{ traverses arc } (i, j), \\ 0, & \text{otherwise.} \end{cases}$$

The optimization problem aims to minimize travel cost over non-working arcs $A_0$, which represents movement without any productive output:

$$\min Z = \sum_{k \in K} \sum_{(i,j) \in A_0} c_{ij}\, x_{ij}^k, \tag{1}$$

where $c_{ij}$ denotes the cost (typically distance) of traversing arc $(i, j)$. Productive in-field arcs $(i, j) \in A_t$ are excluded from the cost summation as they are part of working distance.

**Model Constraints -**

**1. Depot Start and End (per tour).**

$$\sum_{j \in V \setminus \{0\}} x_{0j}^k = 1, \qquad \sum_{i \in V \setminus \{0\}} x_{i0}^k = 1, \quad \forall k \in K. \tag{2}$$

Each route must start and end at the depot. This makes sure that all tours have realistic refill cycles that start and end at the base station. In this formulation, the depot is set to index 0.

**2. Flow Conservation (non-depot vertices).**

$$\sum_{i:\{(i,v) \in A\}} x_{iv}^k = \sum_{j:\{(v,j) \in A\}} x_{vj}^k, \quad \forall v \in V \setminus \{0\},\ \forall k \in K. \tag{3}$$

This ensures that for each visited vertex, the number of incoming arcs is the equal to the number of outgoing arcs within the same tour.

**3. Track Coverage (exactly once).**

$$\sum_{k \in K} \left( x_{2i-1,\,2i}^k + x_{2i,\,2i-1}^k \right) = 1, \qquad \forall i = 1, \ldots, n. \tag{4}$$

Every field track must be serviced exactly once in one direction, ensuring full coverage without overlaps or gaps.

**4. Directional Exclusivity (one-way traversal).**

$$\sum_{k \in K} \left( x_{2i-1,\,2i}^k + x_{2i,\,2i-1}^k \right) \le 1, \qquad \forall i = 1, \ldots, n,\ i \ne j. \tag{5}$$

This prevents simultaneous traversal of both directions of the same connection, which enforces that motion only goes in one direction along each track or headland path.

**5. Capacity Constraint (per tour).**

$$\sum_{i=1}^{n} d_i \left( x^k_{2i-1,\,2i} + x^k_{2i,\,2i-1} \right) \le Q, \qquad \forall k \in K. \tag{6}$$

the total amount of material it needs to carry can't be more than its carrying capacity $Q$. This ensures periodic refilling at the depot when capacity is depleted.

**6. Subtour Elimination (connectivity constraint).**

$$\sum_{\substack{i,j \in S \\ i \neq j}} x^k_{ij} \le |S| - 1, \qquad \forall S \subseteq V \setminus \{0\},\ |S| \ge 2,\ \forall k \in K. \tag{7}$$

These constraints prevent isolated cycles from forming that do not include the depot. This makes sure that all routes are connected and anchored to the depot. A subtour is a cycle over some $S \subseteq V \setminus \{0\}$ that is disconnected from the depot. If a cycle exists on $S$, it uses $|S|$ arcs inside $S$, but (3.7) allows at most $|S| - 1$, so the cycle is cut off. We add these constraints lazily: when an integer solution contains a subtour, we detect its node set $S$ and inject the violated inequality. Because listing all subsets $S$ is exponential, this constraint is not directly added to the model; instead, the solver's lazy constraint mechanism makes sure it is dynamically enforced.

**7. Binary Domain.**

$$x^k_{ij} \in \{0, 1\}, \qquad \forall (i, j) \in A,\ \forall k \in K. \tag{8}$$

All decisions are binary i.e. an arc is either used in a given tour or not.

*Complete Model.* Minimize (1) subject to constraints (2)–(8). This ILP makes sure that each track gets serviced only once during tours that start and end at the depot and are feasible within the limits of the capacity.

*Lazy Constraint Callback in Gurobi -.* In the Gurobi Optimizer, the subtour-elimination constraints (7) are actually used with *lazy constraint callbacks*. Gurobi uses a branch-and-cut framework that includes LP relaxation, branching on integer variables, and the dynamic creation of cutting planes.

The solver starts the search by optimizing a relaxed version of the model that does not have any connectivity constraints. This gives a lower bound on the objective. As the search tree grows, candidate integer solutions (upper bounds) are looked at. The callback routine is called whenever a solution has more than one disconnected subtour. The callback builds the *support graph* consisting of the vertices and the arcs selected in the current incumbent solution (i.e., arcs with $x^k_{ij} = 1$), and then uses depth-first search to detect connected components. Any component $S \subseteq V \setminus \{0\}$ that is disconnected from the depot corresponds to a subtour, so the callback adds the violated cut for that set $S$. Then, it dynamically adds the following cut to make sure that the graph stays connected.

$$\sum_{i,j \in S,\, i \neq j} x^k_{ij} \le |S| - 1$$

This integration of constraint generation into the branch-and-cut process keeps the model compact and accurate because only violated connectivity constraints are ever instantiated. The method is similar to how modern VRP implementations work (e.g., Pessoa *et al.*, 2018) and fits with Gurobi's internal branch-and-cut solver design [13]. So, moderately sized CPP instances can be solved to proven optimality with a lot fewer active constraints than in a static formulation.

## 2.3 Complexity Considerations

The Capacitated Path Planning Problem (CPP) is NP-hard, inheriting complexity from both the CVRP and the CARP [19][21]. The number of possible directed traversals increases by $O(2^n n!)$ for $n$ tracks, making it impossible to do an exhaustive search for anything larger than small cases.

Even with advanced ILP solvers, the runtime grows quickly as $n$ increases. Heuristics and metaheuristics, including Simulated Annealing (SA), Ant Colony Optimization (ACO), and Genetic Algorithms (GA), yield high-quality solutions in polynomial time, sacrificing precision for scalability [10][1]. In this work, the ILP functions as an accurate benchmark for small fields, whereas a customized SA heuristic adapts the methodology to more extensive, practical situations.

# 3 Simulated Annealing Heuristic

Simulated Annealing (SA) provides a scalable, flexible alternative to the exact ILP, trading optimality guarantees for speed and robustness [16]. Our implementation is defined by: (i) a problem-aware *state (solution) representation*; (ii) a set of *neighborhood operators* that perturb the current state; (iii) a capacity-feasible *split/repair* procedure; and (iv) a calibrated *cooling schedule* with an *acceptance criterion* based on a temperature parameter.

Algorithm 1 depicts the simulated annealing-based framework for the capacitated coverage path planning.

A candidate solution is depicted as a *flattened sequence of track visits*, with each visit linked to a direction (forward or reverse). The word "flattened" means that all of the track visits from all of the tours are stored in one long list instead of separate lists for each tour. This representation encodes both routing and sequencing decisions in a compact, order-based form.

During evaluation, the sequence is "split" into separate tours that end at the depot based on the vehicle's capacity $Q$ using a split operator that is capacity-aware. This method makes it easier for the solution operators to generate neighborhoods, whether that means swapping places or moving to a new one, because they can work directly on the linear sequence without having to keep track of tour boundaries. The split procedure rebuilds valid tours and checks capacity and connectivity after each change to make sure that every candidate is still feasible. So, the flattened sequence connects the ILP model's combinatorial structure to the metaheuristic's flexible, permutation-based search space.

In line 3, a greedy heuristic is used to obtain the initial solution [22]. The objective function $C$ evaluates a state by summing only *non-working* travel costs (headland/transition and depot-refill movements) between track endpoints; productive on-track coverage is not penalized, consistent with the formulation in Section 2. Given a candidate move producing state $s'$, the cost difference is $\Delta C = C(s') - C(s)$. SA then applies the standard *Metropolis acceptance rule*: improving moves are accepted, while worsening moves

**Algorithm 1** SA–CPP: Simulated Annealing for Capacitated Coverage Path Planning

**Require:** Cost matrix $C \in \mathbb{R}^{(2n+1)\times(2n+1)}$
Demands $d[1..n]$, capacity $Q$, depot index
Penalty threshold $P$, restart bias $\lambda$
**Ensure:** Best flattened route $R^\star$ and best cost $Z^\star$
1: Define endpoints: $\mathcal{L}(i) \leftarrow 2i-1$, $\mathcal{R}(i) \leftarrow 2i$
2: Define Enter/Exit by direction; Feasible$(u,v) \Leftrightarrow C[u,v] < P$
3: $R \leftarrow$ BuildInitialRoute()
4: $Z \leftarrow$ TotalCost$(R)$
5: $(R^\star, Z^\star) \leftarrow (R, Z)$
6: Estimate $T_0$ by neighbor sampling (target acceptance 0.8)
7: $T \leftarrow T_0$; $T_{\min} \leftarrow \max(10^{-12}, 10^{-4}T_0)$
8: $\alpha \leftarrow 0.999$; $L \leftarrow \max(100, 40n)$
9: **while** $T > T_{\min}$ **and** not stagnated **do**
10: **for** $\ell \leftarrow 1$ to $L$ **do**
11: $R' \leftarrow$ Neighbor$(R)$
12: $Z' \leftarrow$ TotalCost$(R')$ ▷ calls SplitIntoTours$(R', \lambda)$
13: **if** $Z' < \infty$ **then**
14: $\Delta \leftarrow Z' - Z$
15: **if** $\Delta \le 0$ **or** rand$(0,1) < \exp(-\Delta/T)$ **then**
16: $R \leftarrow R'$; $Z \leftarrow Z'$
17: **if** $Z < Z^\star$ **then**
18: $(R^\star, Z^\star) \leftarrow (R, Z)$
19: **end if**
20: **end if**
21: **end if**
22: **end for**
23: $T \leftarrow \alpha T$
24: **end while**
25: **return** $(R^\star, Z^\star)$

may be accepted with probability $\exp(-\Delta C/T)$, where $T$ is the current *temperature*. The temperature is decreased according to the chosen *cooling schedule* (e.g., geometric cooling $T \leftarrow \alpha T$), across a specified number of *main iterations* (temperature updates) and *sub-iterations* (neighbor evaluations at fixed $T$). The algorithm returns the *best-so-far* solution found under the selected stopping criterion.

At each SA iteration, a neighborhood operator is selected uniformly at random (line 11) and applied to the current flattened route $R = \left[(i_1, \mathtt{fwd}_1), \ldots, (i_m, \mathtt{fwd}_m)\right]$, which makes a new candidate route $R'$. The candidate is evaluated by the split function, which reconstructs depot-bounded tours, checks feasibility (capacity and forbidden arcs if any), and computes total non-working cost. Infeasible candidates are discarded. The following operators act directly on the flattened representaion, enabling both local refinement and global structural changes.

(1) **Swap:** exchanges two visits in the sequence, either within or across implicit tours, altering adjacency relations and potentially chaning tour boundaries after splitting.
(2) **Relocate (Or-opt):** removes a short contiguous block (length 1-3) and reinserts it elsewhere, enabling load rebalancing and elimination of inefficient depot returns.
(3) **Reverse:** inverts a contiguous subsequence (2-opt like), modifying adjacency structure and headland transitions to remove zigzagging or directionally inconsistent connections.
(4) **Segment swap (cro-exchannge):** exchanges two equal-length contiguous segments, producing larger structural changes and enabling exploration of alternative depot partitions.
(5) **Flip:** toggles the traversal direction of a selected visit, modifying its entry/exit endpoints without reordering and correcting suboptimal orientation choices.

Together, these operators balance exploitation and exploration. Because all operators act on the flattened sequence, explicit tour boundaries are avoided, and feasibility is consistently enforced by the split operator during evaluation.

Let $C$ denote the total non-working distance of the current solution and $C'$ that of a neighbor. The cost change is $\Delta = C' - C$, and the move is accepted with probability

$$P(\text{accept}) = \begin{cases} 1, & \Delta \le 0, \\ \exp(-\Delta/T), & \Delta > 0, \end{cases}$$

where $T$ is the current *temperature*. Thus, improving moves are always accepted, while worsening ones are accepted with exponentially decreasing probability. This mechanism enables the algorithm to escape local minima early and gradually focus on intensification as $T$ cools [16].

Before the main loop, a short neighbor-sampling phase estimates the initial temperature, $T_0$. A small set of random neighbors is generated from the greedy seed, and the average positive cost increase $\bar{\Delta}$ is measured. We then choose $T_0$ so that such uphill moves would be accepted with probability 0.8:

$$T_0 = -\frac{\bar{\Delta}}{\ln(0.8)}.$$

This data-driven initialization scales the schedule automatically to the instance's cost range and avoids manual tuning.

Temperature decreases geometrically as

$$T_{t+1} = \alpha\, T_t, \qquad \alpha = 0.999,$$

a slow decay that balances exploration and convergence. At each level, $L = \max(100,\ 40n)$ neighbors are evaluated, ensuring sufficient sampling for both small and large fields, where $n$ is the number of tracks.

Annealing stops when either (i) the temperature drops below (see line 7) $T_{\min} = \max(10^{-12},\ 10^{-4}T_0)$ , or (ii) the best cost has not improved over a fixed stagnation window. These criteria prevent unnecessary computation once convergence is evident.

The temperature schedule acts as a control over search randomness: high $T$ values favor diversification by occasionally accepting worse moves, while decreasing $T$ gradually enforces greedier behavior. For the CPP, this behavior is especially beneficial since feasible high-quality routes often lie across narrow ridges of infeasibility; probabilistic acceptance allows the search to cross these barriers and reach better depot partitions and orientations later. The geometric schedule thus provides a predictable, smooth transition from exploration to exploitation and performs robustly across all tested field instances.

The parameters were selected based on preliminary tuning experiments to balance exploration and convergence. While these values performed robustly across all instances, a full sensitivity analysis is left for future work.

For a candidate permutation $R$, the split routine reconstructs explicit depot-bounded tours by scanning the flattened sequence from left to right. Each track in $R$ is appended to the current tour until one of the following conditions triggers a restart at the depot:

(1) **Capacity limit:** This is when adding the next track would cause the accumulated demand to exceed the vehicle capacity $Q$;
(2) **Feasibility failure:** This is when the transition to the entry node of the next track is forbidden or assigned a very large (infeasible) headland cost;
(3) **Restart preference:** This is when returning to the depot and restarting a new tour is sufficiently cheaper than directly continuing to the next track, according to a bias parameter $\beta \in (0, 1]$.

The third rule introduces controlled flexibility in tour partitioning. Formally, when considering a continuation from the current endpoint $p$ to the next entry node $a$, the split routine compares two options:

$$c_{\text{cont}} = C[p, a], \qquad c_{\text{re}} = C[p, \texttt{depot}] + C[\texttt{depot}, a].$$

If either $c_{\text{cont}}$ is infeasible ($\geq \tau$) or $\beta\, c_{\text{re}} < c_{\text{cont}}$, a new tour is started at the depot before visiting $a$. The bias parameter $\beta$ thus controls the algorithm's tendency to restart early:

- When $\beta = 1$, the split behaves greedily where restarts occur only if returning to the depot is *strictly* cheaper than continuing.
- When $\beta < 1$, the routine becomes more restart-prone, slightly favoring depot returns that may lead to shorter overall headland distances after reordering.

In this paper, $\beta$ is treated as a lightweight heuristic control that trades off early restarts (more depot returns) against continuing a partially built tour. We set $\beta = 0.9$ as a conservative value close to the greedy case ($\beta \approx 1$) to avoid excessive fragmentation, while still allowing occasional early restarts when they are near-competitive in cost. In preliminary runs, values substantially below 0.9 tended to produce many short tours, whereas $\beta$ close to 1 reduced restarts but could miss beneficial depot returns. A systematic sensitivity analysis of $\beta$ is, thus, a natural extension and is left for future work.

Empirically, $\beta = 0.9$ provides a good balance between continuity and exploration, avoiding excessively fragmented tours while allowing occasional beneficial restarts.

The cost of a single tour is computed as

$$\begin{aligned} C(\text{tour}) = {} & C[\texttt{depot}, \texttt{Enter}(v_1)] \\ & + \sum_{t=1}^{m-1} C[\texttt{Exit}(v_t), \texttt{Enter}(v_{t+1})] \\ & + C[\texttt{Exit}(v_m), \texttt{depot}]. \end{aligned}$$

where the sum includes only non-working (headland) transitions; working-track movements incur no penalty. The total solution cost is the sum of all tour costs after splitting.

This "giant-tour + split" evaluation pattern decouples sequencing from capacity enforcement: the Simulated Annealing neighborhoods operate on the flattened sequence $R$, while the split function ensures that each evaluated neighbor respects capacity, depot connectivity, and headland feasibility. This design allows SA to explore large permutation spaces efficiently while maintaining consistent operational feasibility during cost evaluation.

## 4 Experimental Setup and Results

This section presents the computational setup, test instances, and comparative results of the experiments. The analyses focus on solution quality, computational efficiency, and scalability across real agricultural fields of increasing complexity.

### 4.1 Test Instances

Three real-world field datasets, anonymized as **Field A**, **Field B**, and **Field C**, were used for evaluation. Table 1 displays the summary of the metrics of said fields while Figure 2 shows the geometric layout of the fields.

Each field was discretized into working tracks and headland transition arcs. Track demands were computed from swath length and the prescribed application rate, while headland arcs model non-productive travel. All depots were located on-field, consistent with typical machinery operations.

The inter-endpoint distance matrix $C[i, j]$ was provided by Verge Ag as part of the field data package. The company's own routing engine figures out the shortest physically possible maneuver between track endpoints in the headland network for each entry. Transitions that were not possible or were not connected were given a large penalty cost ($C[i, j] \geq \tau$) to keep them from being chosen during optimization. This made a symmetric matrix that shows how the field geometry and turning feasibility work in different directions. It is the same input for both the ILP and SA solvers.

Each instance assumes a 48 m working swath, 20 km/h working speed, 15 km/h turning speed, 18 m minimum turn radius, an 8,000 L tank capacity, and an application rate of 125 L/ha. These parameters approximate those of a self-propelled sprayer or liquid fertilizer applicator operating under Controlled Traffic Farming (CTF) conditions.

### 4.2 Methods and Implementations

All algorithms share identical input data: a distance matrix and per-track demand list. The objective for all solvers is the minimization of *non-working distance*, i.e., travel along headland and transfer arcs that do not contribute to field coverage.

*ILP (Python / Gurobi 12.0.0).* The exact method implements the capacitated coverage ILP described in Section 2. Subtour elimination is enforced dynamically via Gurobi's lazy-constraint callback. Default solver parameters were used unless stated otherwise, with time limits applied for large instances.

*Simulated Annealing (C++).* The metaheuristic implementation follows the SA–CPP design of Section 3, using five independent runs

**Table 1: Summary of real-field instances used for evaluation.**

| Field | Perimeter (km) | Area (ha) | Tracks |
|---|---|---|---|
| A | 3.70 | 78.6 | 13 |
| B | 7.26 | 256.6 | 35 |
| C | 12.64 | 409.4 | 96 |

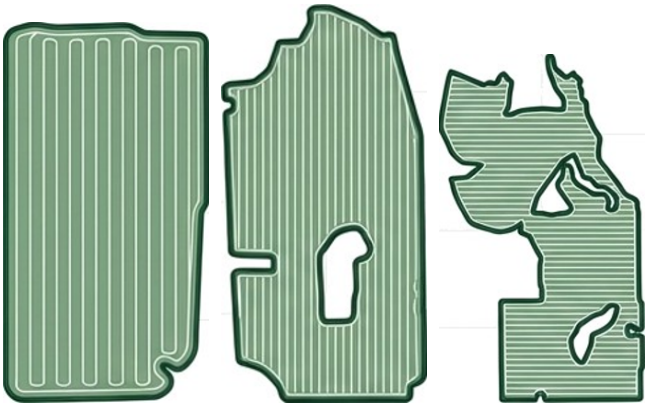


**Figure 2: Representative real-field geometries (Field A, B and C respectively) used in the evaluation, illustrating increasing boundary complexity and internal obstacles.**

per field. Neighborhood operators include `swap`, `2-opt/reverse`, `relocate`, orientation `flip`, and `segment-swap` operators. The temperature schedule uses a geometric cooling rate of 0.999 and a restart bias of 0.9 in the split procedure.

## 4.3 Evaluation Metrics

Algorithmic performance is compared using four metrics:

- **Objective value:** total non-working distance ($Z$).
- **Number of tours:** number of refill trips or capacity-bounded tours.
- **Runtime:** total computational time in seconds.
- **Optimality gap (ILP only):** The relative optimality gap quantifies how close the best integer solution ($Z_{\text{inc}}$) is to the current best bound ($Z_{\text{bd}}$) reported by the solver:

$$\text{Gap} = 100 \times \frac{Z_{\text{inc}} - Z_{\text{bd}}}{|Z_{\text{inc}}|} \ \%.$$

  Here, $Z_{\text{inc}}$ denotes the objective value of the best feasible (incumbent) solution found, while $Z_{\text{bd}}$ represents the best known lower bound on the optimal objective at termination. A smaller gap therefore indicates tighter convergence toward the global optimum.

For the stochastic Simulated Annealing (SA) heuristic, each instance was executed five independent times using identical parameters to assess *consistency* (reproducibility of best-found cost values across runs). This allows quantifying the robustness of the heuristic relative to its inherent randomization.

# 5 Results and Discussion

Table 2 reports five independent SA runs for each field. For each run we record: final best objective (non-working distance), the time at which that best objective was first found, total runtime (time at convergence; see line 9 of Algorithm 1) in seconds, and the number of tours (depot refills) in the final solution. Lower objective values are better. For each field, the numerically best run is highlighted in bold; this run is then used for that field's convergence plot.

**Table 2: SA performance across five runs per field. Bold indicates the best (lowest) objective for each field.**

| Field / Run | Best Cost | Time to Best (s) | Total Runtime (s) | Tours |
|---|---|---|---|---|
| A (Run 1) | 2843 | 6.64 | 10.92 | 2 |
| A (Run 2) | **2747** | 5.86 | 10.63 | 2 |
| A (Run 3) | 2748 | 5.94 | 10.81 | 2 |
| A (Run 4) | 2747 | 4.17 | 9.70 | 2 |
| A (Run 5) | 2843 | 5.26 | 9.99 | 2 |
| B (Run 1) | 10183 | 82.79 | 92.73 | 4 |
| B (Run 2) | **9838** | 66.16 | 75.97 | 4 |
| B (Run 3) | 10807 | 69.67 | 71.98 | 4 |
| B (Run 4) | 10025 | 63.01 | 74.12 | 5 |
| B (Run 5) | 12171 | 48.72 | 73.07 | 4 |
| C (Run 1) | 39279 | 833.4 | 898.1 | 6 |
| C (Run 2) | 36348 | 403.7 | 419.5 | 6 |
| C (Run 3) | 40038 | 754.3 | 766.9 | 6 |
| C (Run 4) | 39488 | 537.3 | 565.6 | 8 |
| C (Run 5) | **36201** | 619.4 | 641.5 | 6 |

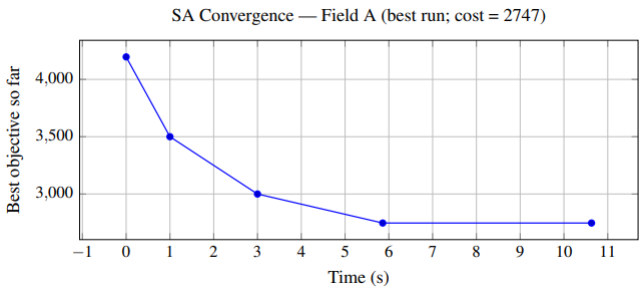
Overall, SA is quite stable as per Table 2: for Field A (13 tracks), four out of five runs are within approximately 3.45% of one another and two runs tie at the global best cost 2747, which matches the ILP optimum. For Field B (35 tracks), all runs finish with costs on the order of $10^4$, which is within 22.00% of each other. The best run (9838) is substantially lower than the rest as well. For Field C (96 tracks), despite being far larger and more topologically complex, SA still converges to costs in the $3.6\times10^4$–$4.0\times10^4$ range, being within about 10.03% of each other, and uses 6–8 depot tours. Moreover, the coefficient of variation (CV) is 1.9%, 9.1%, and 4.8% for Fields A, B, and C respectively, indicating relatively stable performance.

Figures 3a–3c show best-cost-over-time curves for the *best* run in each field. Only the best-so-far objective is plotted. This highlights two behaviors: (i) a steep early drop as SA escapes the naive initial solution, and (ii) a plateau once the algorithm has effectively converged and spends the remainder of its runtime making only neutral or rejected moves.

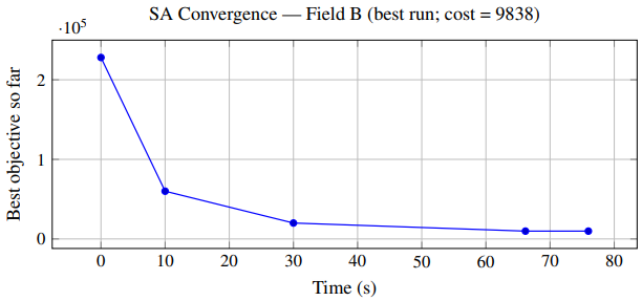
Figure 3a illustrates that the solver very quickly drops from an initial solution of about 4,200 units of non-working distance to 2,747 units, which is the globally optimal value confirmed by the ILP. The best solution is first found at around 6 seconds and never improves further. This shows that SA reaches ILP-quality solutions in practically acceptable time for small fields.

In Figure 3b, the algorithm starts with a very poor initial solution requiring $\sim 2.3\times10^5$ units of non-working distance. Within the first minute it cuts this by more than an order of magnitude and reaches 9,838 units at 66 s. That solution uses 4 tours and is better (lower) than the incumbent found by the ILP under the same instance within a 10,000 s budget. This demonstrates that SA can *outperform* a time-limited ILP on realistic mid-size fields.

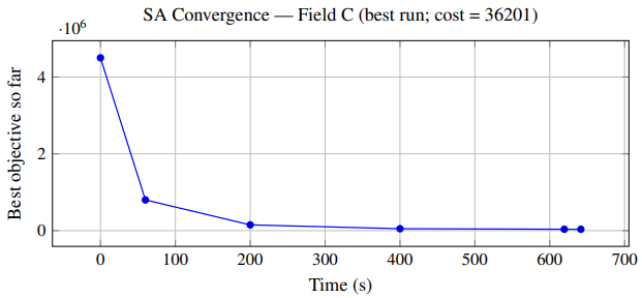
Figure 3c shows the results from the most complex field: 96 tracks, irregular shape, multiple internal islands, and 6–8 depot tours. Even so, SA drives the best-known solution from roughly $4.5\times10^6$ units of non-working distance down to 36,201 units, a $>$ 99% reduction. The best solution is first discovered after about 10 minutes of annealing and then stabilizes. This instance is effectively




**(a) SA convergence for Field A (13 tracks)**




**(b) SA convergence for Field B (35 tracks)**




**(c) SA convergence for Field C (96 tracks)**

**Figure 3: Simulated annealing convergence (best objective so far vs. time) for the best run on each field instance.**

intractable for ILP (the solver stalls with a ∼100% optimality gap even after 10,000 s), so SA is the only method producing a high-quality plan.

Table 3 summarizes the best solutions obtained by Gurobi (ILP), SA, and a proprietary code supplied by Verge AG implementing Ant Colony Optimization (ACO) [10]. For small instances (Field A), all the methods find the optimum nonworking distance. For medium-scale instances (Field B), SA outperforms both ACO and the time-limited Gurobi ILP solver in solution quality while requiring only a fraction of the runtime of Gurobi solver. For large instances (Field C), ILP fails to converge meaningfully, whereas SA continues to provide high-quality feasible routes. SA achieves lower non-working distance than ACO across the tested instances.

The evaluation was performed on a restricted number of real agricultural fields; therefore, the observations must be regarded as context-specific and formal statistical significance could not be assessed.

**Table 3: Best solutions by method (non-working distance, meters).**

| Field | ILP Best | SA Best | ACO |
|---|---|---|---|
| A (13 tracks) | 2747 (optimal) | **2747** | 3947 |
| B (35 tracks) | 9959 (33.5% gap) | **9838** | 10869 |
| C (96 tracks) | $4.33 \times 10^9$ (∼100% gap) | **36201** | 42252 |

- **ILP** is the theoretical standard that is necessary for checking the correctness of a model and finding provably optimal solutions on small to medium-sized problems where computation is possible.
- **SA** is a more useful general-purpose optimizer because it finds almost optimal solutions within short timeframes, even without much parameter tuning across scales.

To confirm that these trends are true in general, more work will need to be done in a wider range of field instances.

From a field operations perspective, these findings translate into tangible efficiency gains. On Field B, SA's best solution (9838 m) not only beats the ILP's time-limited incumbent but also reflects a realistic operational plan with four capacity-based tours. This configuration minimizes both fuel consumption and soil compaction risk. On Field C, SA is the only algorithm capable of producing a complete, high-quality routing plan within a reasonable computational window. This underlines the importance of scalable heuristics for real-world deployment, where problem size and geometric irregularities preclude exact optimization.

## 6 Conclusion and Future Work

This paper presented an integrated framework for *refill-aware, capacitated* in-field route optimization in agricultural operations, emphasizing the minimization of *non-working distance*, which is the travel along headlands or between tracks that does not contribute directly to productive work.

(1) An innovative formulation for capacitated CPP was developed, distinguishing productive (working) arcs from non-productive (headland) arcs and defining a capacitated coverage cost structure. This abstraction unifies the Capacitated Vehicle Routing Problem (CVRP) and Coverage Path Planning (CPP) paradigms [12, 17].
(2) A compact integer linear programming (ILP) model with dynamic subtour elimination via lazy constraints was formulated, providing globally optimal benchmarks for small and medium-sized instances.
(3) A custom Simulated Annealing (SA) metaheuristic with multi-operator neighborhoods (swap, relocate, flip, and segment-swap) and a capacity-aware split procedure was implemented, achieving high-quality solutions in seconds without parameter tuning. The SA consistently matched ILP optima on small fields and surpassed time-limited ILP incumbents on larger cases.

(4) Tests on real-field instances demonstrated scalability from 13 to 96 tracks. SA provided near-optimal routes within minutes, while ILP became computationally intractable.

## 6.1 Future Work

As part of the future work that can be extended from this foundation includes parallelizing the SA solver. Recent improvements in high-performance computing make it possible to speed up metaheuristic search by a lot. Using multicore and GPU hardware to run the Simulated Annealing (SA) process in parallel is a natural next step. Independent annealing threads or neighborhood evaluations can be performed concurrently (e.g., through OpenMP or CUDA), decreasing wall-clock time without compromising search quality. Ferreiro *et al.* (2024) showed that using multiple asynchronous Markov chains with periodic solution exchanges in parallel SA implementations on GPUs can speed things up a lot while still keeping the accuracy of the solutions [11]. If we add to our SA framework in this way, it would allow for near real-time replanning and make it possible to handle large, multi-field routing problems.

## Acknowledgments

This research was partially supported by DIGITAL Super Cluster (Commercializations of Quantum Technologies Co-investment Program with Verge Technologies Inc.), a Mitacs Accelerate Grant, and an Alberta Innovates Graduate Student Scholarship.

## References

[1] Bruce M. Baker and Mulugeta A. Ayechew. 2003. A Genetic Algorithm for the Vehicle Routing Problem. *Computers & Operations Research* 30, 5 (2003), 787–800. doi:10.1016/S0305-0548(02)00051-7

[2] Jonathan F. Bard, George Kontoravdis, and Gang Yu. 2002. A Branch-and-Cut Procedure for the Vehicle Routing Problem with Time Windows. *Transportation Science* 36, 2 (2002), 250–269. doi:10.1287/trsc.36.2.250.563

[3] Dimitrios Bochtis and Claus Sørensen. 2010. The Vehicle Routing Problem in Field Logistics — Part II. *Biosystems Engineering* 105, 2 (2010), 180–188. doi:10.1016/j.biosystemseng.2009.11.006

[4] Dimitrios Bochtis, Claus Sørensen, and Simon Green. 2009. The Vehicle Routing Problem in Field Logistics — Part I. *Biosystems Engineering* 104, 4 (2009), 447–457. doi:10.1016/j.biosystemseng.2009.09.003

[5] Kris Braekers, Katrien Ramaekers, and Inneke Van Nieuwenhuyse. 2016. The Vehicle Routing Problem: State of the Art Classification and Review. *Computers & Industrial Engineering* 99 (2016), 300–313. doi:10.1016/j.cie.2015.12.007

[6] Y. Chen. 2023. Approximation Schemes for Capacitated Vehicle Routing Problems: A Survey. *Applied and Computational Mathematics* 22 (2023), 1–27. doi:10.48550/arXiv.2306.01826

[7] Howie Choset. 2001. Coverage for Robotics: A Survey of Recent Results. *Annals of Mathematics and Artificial Intelligence* 31, 1–4 (2001), 113–126. doi:10.1023/A:1016639210559

[8] Ángel Corberán and Gilbert Laporte. 2015. *Arc Routing: Problems, Methods, and Applications.* SIAM, Philadelphia, PA, USA. doi:10.1137/1.9781611973679

[9] George B. Dantzig and John H. Ramser. 1959. The Truck Dispatching Problem. *Management Science* 6, 1 (1959), 80–91. doi:10.1287/mnsc.6.1.80

[10] Marco Dorigo and Thomas Stützle. 2004. *Ant Colony Optimization.* MIT Press, Cambridge, MA, USA.

[11] A. M. Ferreiro, J. A. García, J. G. López-Salas, and C. Vázquez. 2024. An Efficient Implementation of Parallel Simulated Annealing Algorithm in GPUs. arXiv. arXiv:2408.00018 [cs.DC] https://arxiv.org/abs/2408.00018

[12] Enric Galceran and Marc Carreras. 2013. A Survey on Coverage Path Planning for Robotics. *Robotics and Autonomous Systems* 61, 12 (2013), 1258–1276. doi:10.1016/j.robot.2013.09.004

[13] Gurobi Optimization, LLC. 2024. *Gurobi Optimizer Reference Manual.* https://www.gurobi.com/documentation/ Accessed: 2026-01-24.

[14] M. Höffmann, S. Patel, and C. Büskens. 2023. Optimal Coverage Path Planning for Agricultural Vehicles with Curvature Constraints. *Agriculture* 13, 11 (2023).

[15] Amin Khajepour, Majid Sheikhmohammady, and Ehsan Nikbakhsh. 2020. Field Path Planning Using Capacitated Arc Routing Problem. *Computers and Electronics in Agriculture* 173, Article 105401 (2020). doi:10.1016/j.compag.2020.105401

[16] Scott Kirkpatrick, C. Daniel Jr. Gelatt, and Mario P. Vecchi. 1983. Optimization by Simulated Annealing. *Science* 220, 4598 (1983), 671–680. doi:10.1126/science.220.4598.671

[17] Tuomas Oksanen and Arto Visala. 2009. Coverage Path Planning Algorithms for Agricultural Field Machines. *Journal of Field Robotics* 26, 8 (2009), 651–668. doi:10.1002/rob.20300

[18] A. Pessoa, M. Poggi, A. Subramanian, and E. Uchoa. 2018. Branch-and-Cut-and-Price for the Robust Capacitated Vehicle Routing Problem. Optimization Online. http://www.optimization-online.org/DB_HTML/2018/08/6762.html Accessed: 2026-01-24.

[19] Paolo Toth and Daniele Vigo. 2002. *The Vehicle Routing Problem.* SIAM, Philadelphia, PA, USA. doi:10.1137/1.9780898718515

[20] M. Vahdanjoo, K. Zhou, and Claus A. G. Sørensen. 2020. Route Planning for Agricultural Machines with Multiple Depots: Manure Application Case Study. *Agronomy* 10, 10 (2020), 1608. doi:10.3390/agronomy10101608

[21] Sanne Wøhlk. 2008. A Decade of Capacitated Arc Routing. In *The Vehicle Routing Problem: Latest Advances and New Challenges*, Bruce Golden, S. Raghavan, and Edward Wasil (Eds.). Springer, Boston, MA, USA, 29–48. doi:10.1007/978-0-387-77778-8_2

[22] Fabliha Zahin. 2025. *High-performance exact and metaheuristic methods for agricultural path planning.* Master's thesis. University of Lethbridge, Lethbridge, Alberta, Canada. https://hdl.handle.net/10133/7282 Master's thesis.